EEN025 Assignment 2

Group 6

October 7, 2026

Task 1: The classical Job Shop Problem

The classical Job Shop Scheduling Problem consists of a number of jobs and machines. Each job contains a fixed sequence of operations. Every operation must be processed on a specified machine for a given amount of time. The order of operations within each job must be respected, and each machine can process at most one operation at a time.

The objective is to minimize the makespan, which is the time at which all jobs have been completed.

MILP Formulation

Let (s_o) denote the starting time of operation (o), and let (p_o) denote its processing time. The makespan is denoted by ($C_{\max}$).

The objective function is:

$$\min C_{\max}$$

For two consecutive operations (o) and (o') belonging to the same job, the precedence constraint is:

$$s_{o'} \geq s_o + p_o$$

This constraint ensures that an operation cannot start before the previous operation of the same job has been completed.

If two operations use the same machine, they cannot overlap. A binary variable ($y_{o,o'}$) is introduced to determine their order:

$$s_o + p_o \leq s_{o'} + M(1 - y_{o,o'})$$

$$s_{o'} + p_{o'} \leq s_o + M y_{o,o'}$$

where ($y_{o,o'} \in 0, 1$) and (M) is a sufficiently large constant. When ($y_{o,o'} = 1$), operation (o) is scheduled before (o'). When ($y_{o,o'} = 0$), the opposite order is selected.

For the last operation (l_j) of each job (j), the makespan constraints are:

$$C_{\max} \geq s_{l_j} + p_{l_j}$$

The MILP model was implemented and solved using Gurobi.

Constraint Programming formulation

The CP model uses the same start-time variables and precedence constraints as the MILP model. However, the machine-capacity constraint is expressed directly as a logical disjunction:

$$(s_o + p_o \leq s_{o'}) \vee (s_{o'} + p_{o'} \leq s_o)$$

Thus, either operation (o) is completed before operation (o') starts, or the reverse is true.

The objective is again:

$$\min C_{\max}$$

The CP model was implemented and solved using Z3.

Experimental Results

The models were tested on four Job Shop benchmark instances: `ft06`, `ft10`, `la01`, and `la05`. The time limit was set to 1200 seconds for each solver. The results are shown in Table 1.

Table 1: Experimental results for the MILP and CP models.

Instance	Solver	Status	Makespan	Runtime (s)
`ft06`	Gurobi	Optimal	55	0.0361
`ft06`	Z3	SAT	55	0.0441
`ft10`	Gurobi	Optimal	930	6.4151
`ft10`	Z3	SAT	930	28.3073
`la01`	Gurobi	Optimal	666	0.1380
`la01`	Z3	SAT	666	0.8890
`la05`	Gurobi	Optimal	593	0.2864
`la05`	Z3	SAT	593	5.8442

Both formulations obtained the same makespan for all tested instances. Gurobi was faster than Z3 in every case. The difference was particularly clear for `ft10`, where Gurobi required 6.4151 seconds compared with 28.3073 seconds for Z3.

Task 2: The Extended Job Shop Problem

Transportation graph

In the classical Job Shop Problem, transportation time between two consecutive machines is assumed to be zero. In this task, this assumption is removed. The shop floor is represented as a weighted graph, where the nodes represent machines, the warehouse, the delivery area and intermediate junctions. The edges represent possible transportation routes, and their weights are the corresponding distances in metres.

The transportation speed is assumed to be constant and equal to 5 metres per second. If the shortest distance between two locations a and b is $d_{a,b}$, the transportation time is calculated as $t_{a,b}^{\text{travel}} = \frac{d_{a,b}}{5}$.

A* algorithm

The A* algorithm selects the node with the smallest estimated total cost $f(n) = g(n) + h(n)$. The value $g(n)$ represents the actual distance from the start node to the current node, while $h(n)$ estimates the remaining distance to the goal. Whenever a shorter route to a neighbor is found, its cost and predecessor are updated. Once the goal is selected, the path is reconstructed by following the predecessor pointers backwards.

Algorithm 1 A* algorithm

```
Initialize Open with the start node and set g(s) = 0
while Open is not empty do
    Select the node with minimum f(n) = g(n) + h(n)
    if the selected node is the goal then
        Return the reconstructed path
    end if
    Update the costs and parents of all neighbours
    Add improved neighbours to Open
end while
Return failure
```

Shortest-path results

The shortest distances obtained by A* are shown in Table 2. The entries are distances in metres. The corresponding transportation time can be obtained by dividing each entry by 5.

Table 2: Shortest distances between the main locations in the shop floor.

From/To	W	M1	M2	M3	M4	M5	M6	D
W	0	90	70	142	64	28	155	161
M1	90	0	54	52	64	62	65	71
M2	70	54	0	72	118	42	119	91
M3	142	52	72	0	116	114	81	53
M4	64	64	118	116	0	78	91	135
M5	28	62	42	114	78	0	127	133
M6	155	65	119	81	91	127	0	100
D	161	71	91	53	135	133	100	0

W denotes the warehouse and D denotes the delivery area.

The distance matrix is symmetric because the transportation routes were modelled as bidirectional.

Extended Job Shop formulation

The classical Job Shop model assumes zero transportation time between machines. In the extended model, the transportation time obtained from the A* algorithm is added to the precedence constraints.

Let $s_{j,h}$ be the start time of operation h of job j, $p_{j,h}$ its processing time, and $m_{j,h}$ its assigned machine. The precedence constraint becomes

$$s_{j,h+1} \geq s_{j,h} + p_{j,h} + t^{\text{travel}}_{m_{j,h},m_{j,h+1}},$$

where

$$t^{\text{travel}}_{a,b} = \frac{d_{a,b}}{5}.$$

The first operation also includes transportation from the warehouse:

$$s_{j,0} \geq t^{\text{travel}}_{\text{warehouse},m_{j,0}}.$$

After the final operation, the job must reach the delivery area:

$$C_{\max} \geq s_{j,l_j} + p_{j,l_j} + t^{\text{travel}}_{m_{j,l_j},\text{delivery}}.$$

The machine-capacity constraints remain the same as in Task I. Thus, the extended model minimizes the makespan while considering both processing and transportation times.

Extended ft06 result

After incorporating the transportation times, the extended model obtained an optimal makespan of

$$C^{\text{extended}}_{\max} = 177.4 \text{ s}.$$

The original ft06 model without transportation time had a makespan of 55 seconds. Therefore, the increase in makespan was

$$\Delta C_{\max} = 177.4 - 55 = 122.4 \text{ s}.$$

The result shows that transportation time can have a significant effect on the total completion time, especially when jobs must travel long distances between consecutive machines or to the delivery area.

Task 3: Finding the Shortest Path using Z3

The objective of this task is to find 10 shortest path between machine 5 and machine 3. Therefore, following the SAT-based shortest path formulation of Aloul et al, We define one boolean variable per edge, E_v and one per node N_v. If $E_v = True$ or $N_v = True$, it means the edge or the node is a part of the selected path.

Graph

The shop floor is modelled as a 5×8 grid, with non-existent connections removed to match the actual layout in Figure 1. Nodes are identified by $(row, column)$ coordinates, and machines are placed at their corresponding nodes.

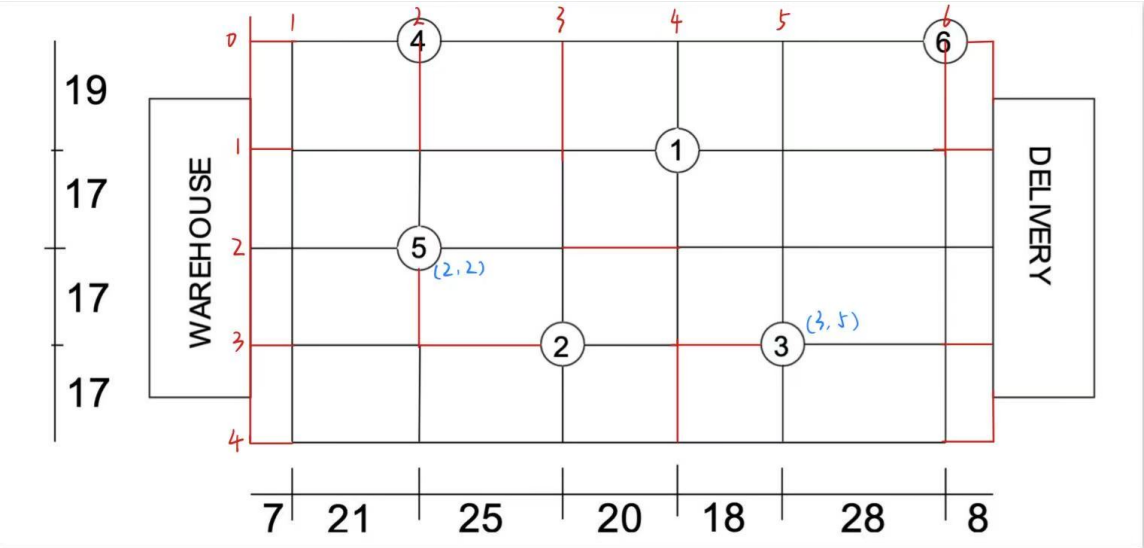


Figure 1: The coordination graph

Constraints

If one edge $e = (u, v)$ is selected, the two nodes u and v connected by this edge are also labelled as selected. Transformed to clausal constraint is

$$E_e \Rightarrow N_u, \qquad E_e \Rightarrow N_v \qquad \forall e = (u, v)$$

Then, the selected path starts from the beginning point s, from which only one edge can be chosen; the same requirement holds for the destination node t.

$$\bigvee_{e \ni s} E_e, \qquad \bigwedge_{\{e_1, e_2\} \subseteq \delta(s)} \neg(E_{e_1} \wedge E_{e_2})$$

For every other node v, if it is not on the path, we force it to have 0 incident edges. If it is on the selected path, is has only 2 incident, one in and one out.

$$\bigwedge_{\{e_1, e_2, e_3\} \subseteq \delta(v)} \neg(E_{e_1} \wedge E_{e_2} \wedge E_{e_3})$$

When the point on the path, then for every incident edge e_k, at least one of the other incident edges must also be selected.

$$N_v \Rightarrow \bigvee_{e \in \delta(v) \setminus \{e_k\}} E_e \quad \text{for every } e_k \in \delta(v)$$

.

During the simulation, we found that although z3 solver gives the optimal solutions followed the above constraints, there are some unideal solutions included, the cycles.

To detect this, we walk along the selected edges from the start node to the end node. The edges visited during this walk form the actual path; any remaining selected edges that were not visited must form one or more disconnected cycles. If such cycles exist, we add a constraint forbidding only that specific set of cycle edges from being selected again. If no cycle is found, the solution is accepted as a valid path, and a separate constraint excludes that exact path from future solutions.

This process is repeated, each iteration either eliminates a cycle or yields one valid path, until either the solver returns UNSAT or the required number of shortest 10 paths has been collected.

Result

The results of this task are illustrated in Tab3.

Table 3: 10 shortest paths from Machine 5 (2,2) to Machine 3 (3,5)

Rank	**Length (m)**	**Path (grid coordinates)**
1	114	(2,2) → (1,2) → (1,3) → (1,4) → (2,4) → (2,5) → (3,5)
2	114	(2,2) → (1,2) → (1,3) → (1,4) → (1,5) → (2,5) → (3,5)
3	114	(2,2) → (2,3) → (3,3) → (3,4) → (2,4) → (2,5) → (3,5)
4	114	(2,2) → (2,3) → (1,3) → (1,4) → (1,5) → (2,5) → (3,5)
5	114	(2,2) → (2,3) → (3,3) → (4,3) → (4,4) → (4,5) → (3,5)
6	114	(2,2) → (2,3) → (1,3) → (1,4) → (2,4) → (2,5) → (3,5)
7	148	(2,2) → (1,2) → (1,3) → (2,3) → (3,3) → (4,3) → (4,4) → (4,5) → (3,5)
8	148	(2,2) → (1,2) → (1,3) → (2,3) → (3,3) → (3,4) → (2,4) → (2,5) → (3,5)
9	148	(2,2) → (2,3) → (3,3) → (3,4) → (2,4) → (1,4) → (1,5) → (2,5) → (3,5)
10	152	(2,2) → (1,2) → (1,3) → (1,4) → (0,4) → (0,5) → (1,5) → (2,5) → (3,5)

The path 1-6 are the shortest path, the distance is $114m$.